

Compact Raw-IMU Activity Recognition on UCI HAR

Compact Raw-IMU Activity Recognition on UCI HAR

Take-home exercise, research assistantship, Embedded Systems Laboratory (ESL), EPFL.

Task. Train a neural network with **fewer than 20,000 trainable parameters** that classifies human activity from a window of **raw** accelerometer and gyroscope signals (no handcrafted features). Assess it with accuracy, F1 and a confusion matrix, and discuss strengths, limitations and failure cases.

What was built. Three hand-designed 1D CNNs (**Models A, B, C**) were compared on a subject-held-out validation split. Model C was selected and retrained on all official training subjects for a fixed 17 epochs. Afterwards a fourth, **strictly causal** network (**Model D**) and a linear baseline were trained as a validation-only extension, to place all models on an accuracy-vs-cost **Pareto** plot.

Headline result (official test set, 9 unseen subjects, 2,947 windows, Model C, 17 epochs, 8,862 parameters): accuracy **0.9240**, macro-F1 **0.9231**.

1. Data and preprocessing

UCI HAR: 30 subjects, six activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING), smartphone on the waist, 50 Hz, pre-segmented into 128-sample windows (2.56 s, 50 % overlap). The official split is subject-independent: 21 training subjects (7,352 windows) and 9 test subjects (2,947 windows).

- **Raw signals only.** Input is (6, 128) per window: total_acc_x/y/z and body_gyro_x/y/z. The precomputed 561-feature files (X_train.txt, X_test.txt) are never opened; the loader refuses to read them and this is covered by a unit test.
- **total_acc rather than body_acc.** Gravity direction is the main cue separating the static postures. The sensor ablation in Section 6 supports this choice.
- **Validation = held-out subjects, not windows.** Windows overlap by 50 %, so a window-level split would put near-duplicates on both sides. Four of the 21 training subjects (1, 3, 25, 27) are held out for model selection; the other 17 train.
- **Normalisation.** Per-channel z-score, statistics computed on training data only. For the final model they are recomputed on all 21 training subjects and stored in the checkpoint; the test set is only ever transformed with them.
- **Test-set discipline.** The test split is never loaded by training or by any selection step.

2. Models

All networks follow Conv1D -> BatchNorm -> ReLU -> (pool) -> ... -> Global Average Pooling -> Linear. The architectures are declared once in src/architectures.py, and parameter and MAC counts are computed from that single specification (and cross-checked against hooks on the real PyTorch module).

2.1 Models A, B, C (the primary comparison)

Model	Idea	Params	MAC / window	Local receptive field
A	Minimal: 2 conv layers (6->16 k9, 16->32 k5), 1 pooling	3,766	274,624	18 samples (0.36 s)
B	3 dense conv layers (16/32/64 ch), 2 pooling stages	14,390	602,496	36 samples (0.72 s)
C	Depthwise-separable blocks (k9 depthwise + 1x1 pointwise), 24/48/96 ch, 2 pooling stages	8,862	415,296	60 samples (1.20 s)

- **A** is the "how little is enough?" floor.
- **B** adds depth with full convolutions: more capacity, larger receptive field, but the most parameters and compute.
- **C** swaps dense convolutions for depthwise-separable ones. It reaches a receptive field 67 % larger than B's with 38 % fewer parameters and 31 % fewer MACs, because it never forms a large $C_{in} \times C_{out} \times K$ kernel.

2.2 Model D (the causal extension, trained after A/B/C were frozen)

A, B and C use symmetric ("same") padding, so each output depends on *future* samples and cannot run sample-by-sample on a stream. **D** was designed to remove that obstacle:

- four stages of **dilated, depthwise-separable, left-padded (causal)** convolutions, kernel 5, dilations 1/2/4/8, channels 6->16->24->32->48;
- **no pooling**, so every layer has a fixed-size history buffer;
- 3,906 parameters, 406,560 MACs/window, receptive field 61 samples, then GAP + Linear.

D was specified once from first principles before any validation number existed and was not tuned afterwards. A **Linear** baseline (Linear(768, 6) on the flattened window, 4,614 parameters) tests whether nonlinear temporal modelling is needed at all.

All five models are far below the 20,000-parameter limit.

```
import json, sys
from pathlib import Path
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from IPython.display import display

ROOT = Path.cwd()
if not (ROOT / "results").exists() and (ROOT.parent / "results").exists():
    ROOT = ROOT.parent
M = ROOT / "results" / "metrics"
EXT = ROOT / "results" / "research_extension"
load = lambda p: json.load(open(p))
sel = load(M / "architecture_selection.json")["summary"]
ext = load(EXT / "metrics" / "research_extension_summary.json")
final = load(M / "final_test_evaluation.json")
pred = pd.read_csv(M / "final_test_predictions.csv")
cx = ext["complexity"]
print("parameter budget check (must be < 20,000):")
for k, v in cx.items():
    print(f" {k:6s} {v['params']:6,d} params {'OK' if v['params'] < 20000 else 'TOO BIG'})"
```

parameter budget check (must be < 20,000):		
Linear	4,614 params	OK
D	3,906 params	OK
A	3,766 params	OK
B	14,390 params	OK
C	8,862 params	OK

3. Training protocol

Phase 1 - architecture comparison (validation only). A, B, C x 3 seeds (0, 1, 2) on the fixed 17/4-subject split with identical settings: Adam (lr 1e-3, weight decay 1e-4), batch 64, cross-entropy, up to 100 epochs, early stopping (patience 15) and ReduceLRonPlateau on validation macro-F1, best-validation-F1 checkpoint. The only things that vary are architecture and seed. The winner is the highest *mean* validation macro-F1, a rule fixed beforehand.

Phase 2 - final model. The selected architecture is retrained from scratch (seed 123) on all 7,352 training windows with no validation set, for a fixed budget round(mean(epochs_ran)) = round(mean([17, 17, 18])) = 17 epochs, and the last-epoch weights are saved. The budget comes only from Phase-1 training durations.

Single test evaluation. The 17-epoch checkpoint was then evaluated once on the official test set (Section 5); the evaluation script refuses to overwrite an existing result.

Provenance note. An earlier draft of Phase 2 used the epoch of best validation F1 (1 epoch) and was evaluated on the test set before the budget rule was changed to the training-duration rule. That checkpoint is **not** the submitted model and none of its numbers appear below; it is archived under results/historical_original_protocol/ only for transparency. Because that earlier evaluation had been seen, the test number here should be read as a one-shot result of a rule that was fixed from Phase-1 behaviour, not as a blind first look.

4. Validation results and Pareto analysis

```
names = ["Linear", "A", "B", "C", "D"]
fmean = {"A": sel["A"]["mean_val_macro_f1"], "B": sel["B"]["mean_val_macro_f1"], "C": sel["C"]["mean_val_macro_f1"],
         "D": ext["model_d"]["mean_val_macro_f1"], "Linear": ext["linear_baseline"]["mean_val_macro_f1"]}
fstd = {"A": sel["A"]["std_val_macro_f1"], "B": sel["B"]["std_val_macro_f1"], "C": sel["C"]["std_val_macro_f1"],
        "D": ext["model_d"]["std_val_macro_f1"], "Linear": ext["linear_baseline"]["std_val_macro_f1"]}
rows = []
for n in names:
    rows.append({"Model": n, "Val macro-F1 (mean)": round(fmean[n], 4), "Std (3 seeds)": round(fstd[n], 4),
                "Params": cx[n]["params"], "MAC/window": cx[n]["macs"], "Receptive field": cx[n]["receptive_field"],
                "Causal": "yes" if cx[n]["causal"] else "no"})
tab = pd.DataFrame(rows).set_index("Model")
display(tab)
```

	Val macro-F1 (mean)	Std (3 seeds)	Params	MAC/window	Receptive field	Causal
Model						
Linear	0.6203	0.0013	4614	4608	128	no
A	0.9890	0.0006	3766	274624	18	no
B	0.9813	0.0007	14390	602496	36	no
C	0.9898	0.0005	8862	415296	60	no
D	0.9778	0.0062	3906	406560	61	yes

Validation macro-F1 is measured on four held-out training subjects (3 seeds each). C is the best by the pre-set rule (0.9898), A is within 0.0008 of it, B is clearly worse despite 4x the parameters of A, D is slightly lower and noisier, and the Linear model is far behind.

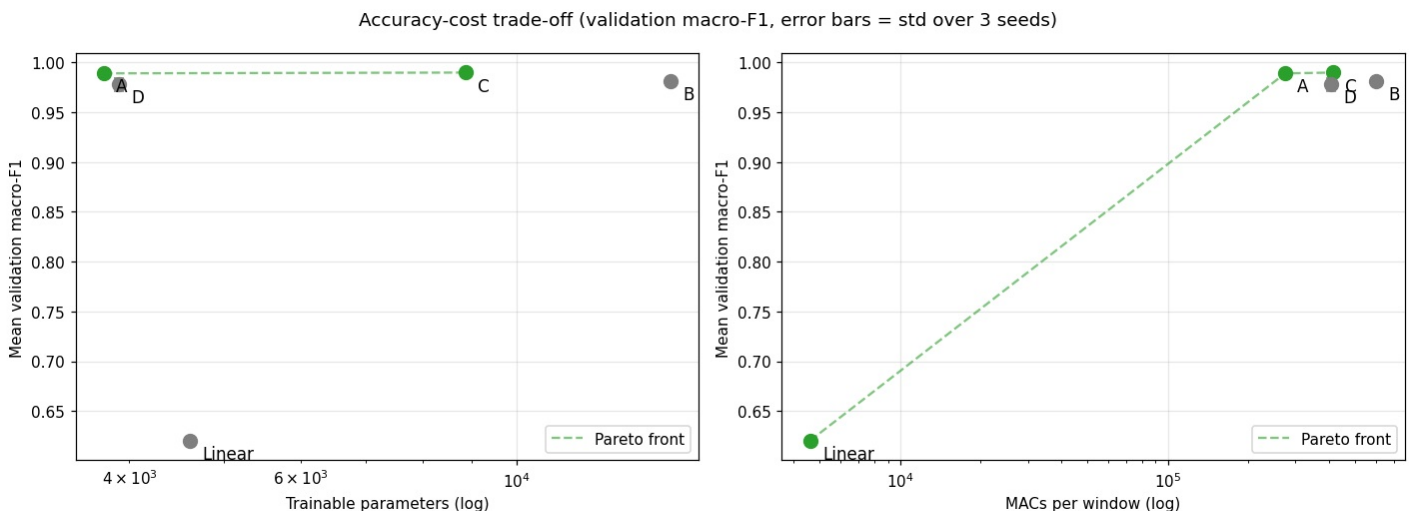
Pareto frontier: accuracy against cost

A model is **dominated** if another model is at least as accurate *and* at most as costly, with at least one strict inequality.

```
def dominated(metric, cost):
    out = {}
    for a in names:
        by = [b for b in names if b != a and metric[b] >= metric[a] and cost[b] <= cost[a]
              and (metric[b] > metric[a] or cost[b] < cost[a])]
        out[a] = by
    return out

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
for ax, key, lab in ((axes[0], "params", "Trainable parameters"), (axes[1], "macs", "MACs per window")):
    cost = {n: cx[n][key] for n in names}
    dom = dominated(fmean, cost)
    front = sorted([n for n in names if not dom[n]], key=lambda n: cost[n])
    for n in names:
        ax.errorbar(cost[n], fmean[n], yerr=fstd[n], fmt="o", ms=9, capsize=3,
                    color="tab:green" if not dom[n] else "tab:gray")
        ax.annotate(n, (cost[n], fmean[n]), textcoords="offset points", xytext=(8, -12), fontsize=11)
    ax.plot([cost[n] for n in front], [fmean[n] for n in front], "--", color="tab:green", alpha=.6, label="Pareto front")
    ax.set_xscale("log"); ax.set_xlabel(lab + " (log)"); ax.set_ylabel("Mean validation macro-F1")
    ax.grid(alpha=.3); ax.legend(loc="lower right")
    print(lab, "->", {n: ("dominated by " + ", ".join(dom[n])) if dom[n] else "non-dominated" for n in names})
fig.suptitle("Accuracy-cost trade-off (validation macro-F1, error bars = std over 3 seeds)")
fig.tight_layout(); plt.show()
```

```
Trainable parameters -> {'Linear': 'dominated by A, D', 'A': 'non-dominated', 'B': 'dominated by A, C', 'C': 'non-dominated',
'D': 'dominated by A'}
MACs per window -> {'Linear': 'non-dominated', 'A': 'non-dominated', 'B': 'dominated by A, C', 'C': 'non-dominated', 'D':
'dominated by A'}
```



Reading the Pareto plot (validation only, 4 subjects, 3 seeds, so these are indications rather than significance claims):

- **A** and **C** are on the front. A is the cheapest accurate model; C buys +0.0008 macro-F1 (inside seed noise) for 2.4x the parameters and 1.5x the MACs.
- **B** is dominated by A (and by C): it costs more and scores lower, so its extra capacity was not usable at this data scale.
- **D** is dominated by A. This is the price of causality and of no pooling (no MAC savings from downsampling). D is useful as a streaming-friendly design point, not as the accuracy winner.
- **Linear** is non-dominated only in the MAC view, because it is nearly free (4,608 MACs); at 0.62 macro-F1 it is not a usable classifier, which confirms that nonlinear temporal feature extraction is required.
- In the parameter view, C and A are the only non-dominated models.

Why C was still the one submitted. The selection rule (highest mean validation macro-F1) was fixed before the runs. A is the natural choice if cost matters more than a 0.0008 difference, and is flagged as the embedded candidate in Section 7.

5. Final model: official test-set evaluation

Model C, 8,862 parameters, trained 17 epochs on all 21 training subjects (seed 123), evaluated **once** on the 9 test subjects.

```
cn = final["class_names"]
print(f"accuracy      : {final['accuracy']:.4f}")
print(f"macro F1       : {final['macro_f1']:.4f}")
print(f"weighted F1      : {final['weighted_f1']:.4f}    (n = {final['n_samples']} windows, 9 unseen subjects)\n")
pc = pd.DataFrame(final["per_class"]).set_index("name")[["precision", "recall", "f1", "support"]].round(4)
display(pc)
```

```

cm = np.array(final["confusion_matrix"])
fig, axes = plt.subplots(1, 2, figsize=(13, 5.3))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, square=True, xticklabels=cn, yticklabels=cn, ax=axes[0])
axes[0].set_title("Confusion matrix (counts)")
sns.heatmap(cm / cm.sum(1, keepdims=True), annot=True, fmt=".2f", cmap="Blues", cbar=False, square=True,
            xticklabels=cn, yticklabels=cn, ax=axes[1])
axes[1].set_title("Row-normalised (recall)")
for ax in axes:
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.tick_params(axis="x", rotation=45)
fig.suptitle("Model C (17 epochs) on the official test set"); fig.tight_layout(); plt.show()

```

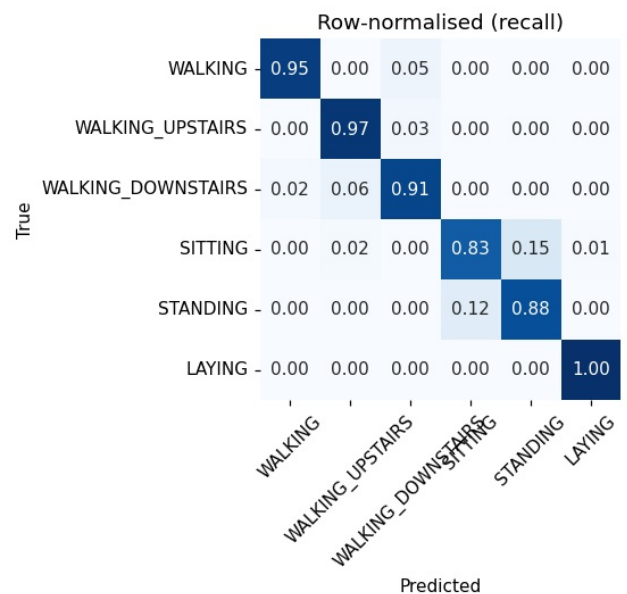
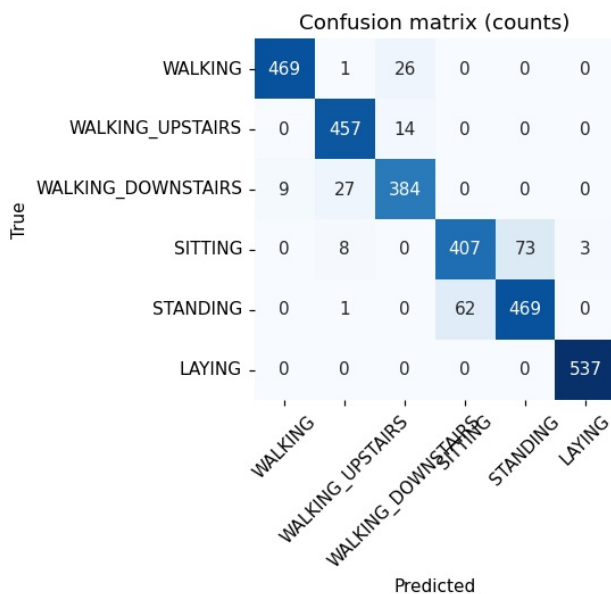
```

accuracy      : 0.9240
macro F1      : 0.9231
weighted F1   : 0.9238    (n = 2947 windows, 9 unseen subjects)

```

	precision	recall	f1	support
name				
WALKING	0.9812	0.9456	0.9630	496
WALKING_UPSTAIRS	0.9251	0.9703	0.9472	471
WALKING_DOWNSTAIRS	0.9057	0.9143	0.9100	420
SITTING	0.8678	0.8289	0.8479	491
STANDING	0.8653	0.8816	0.8734	532
LAYING	0.9944	1.0000	0.9972	537

Model C (17 epochs) on the official test set



6. Strengths, limitations and failure cases

```

gb = final["activity_group_breakdown"]
tot = gb["total_errors"]
print(f"{tot} of {final['n_samples']} windows misclassified")
print(f"  within static (SITTING/STANDING/LAYING): {gb['within_group_errors']['static']:3d}
({100*gb['within_group_fraction']['static']:.1f}% of errors)")
print(f"  within dynamic (three walking variants) : {gb['within_group_errors']['dynamic']:3d}
({100*gb['within_group_fraction']['dynamic']:.1f}% of errors)")
print(f"  across static->dynamic : {gb['across_group_errors']:3d} ({100*gb['across_group_fraction']:.1f}%
of errors)\n")
print("most confused pairs (true -> predicted):")
for p in final["most_confused_pairs"][:6]:
    print(f"  {p['true']:<19} -> {p['predicted']:<19} {p['count']:3d} ({100*p['fraction_of_true_class']:.1f}% of that
class)")

ps = pred.groupby("subject_id").agg(windows=("correct", "size"), accuracy=("correct", "mean"))
ps["errors"] = (ps.windows * (1 - ps.accuracy)).round().astype(int)
display(ps.round(3).sort_values("accuracy"))
fig, ax = plt.subplots(figsize=(8, 3.4))
ps.sort_index().accuracy.plot.bar(ax=ax, color="tab:blue"); ax.axhline(final["accuracy"], color="k", ls="--",
label="overall")
ax.set_ylim(0.6, 1.02); ax.set_ylabel("Accuracy"); ax.set_title("Test accuracy per subject"); ax.legend(); plt.show()

```

```
worst = pred[pred.subject_id.isin(ps.accuracy.nsmallest(3).index) & ~pred.correct]
print("errors made by the 3 worst subjects:", len(worst), "of", tot)
print("SITTING<->STANDING errors from subject 10 alone:",
      len(pred[(pred.subject_id == 10) & ~pred.correct & pred.true_name.isin(["SITTING", "STANDING"]) &
pred.pred_name.isin(["SITTING", "STANDING"])]))
```

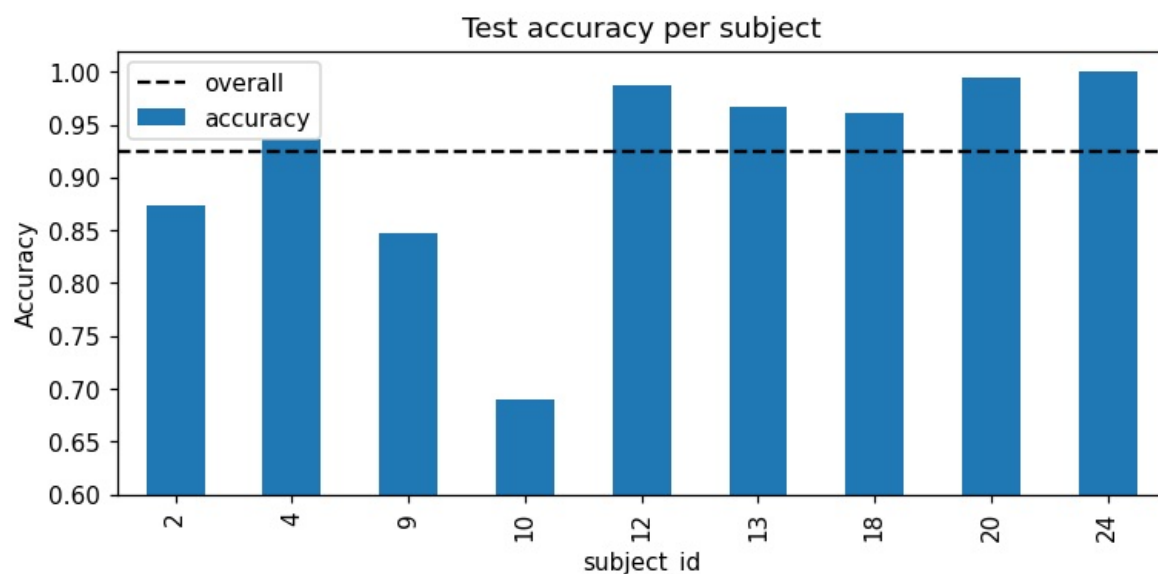
224 of 2947 windows misclassified
 within static (SITTING/STANDING/LAYING): 138 (61.6% of errors)
 within dynamic (three walking variants) : 77 (34.4% of errors)
 across static<->dynamic : 9 (4.0% of errors)

most confused pairs (true -> predicted):

SITTING	-> STANDING	73	(14.9% of that class)
STANDING	-> SITTING	62	(11.7% of that class)
WALKING_DOWNSTAIRS	-> WALKING_UPSTAIRS	27	(6.4% of that class)
WALKING	-> WALKING_DOWNSTAIRS	26	(5.2% of that class)
WALKING_UPSTAIRS	-> WALKING_DOWNSTAIRS	14	(3.0% of that class)
WALKING_DOWNSTAIRS	-> WALKING	9	(2.1% of that class)

errors made by the 3 worst subjects: 173 of 224
 SITTING<->STANDING errors from subject 10 alone: 48

	windows	accuracy	errors
subject_id			
10	294	0.690	91
9	288	0.847	44
2	302	0.874	38
4	317	0.937	20
18	364	0.962	14
13	327	0.966	11
12	320	0.988	4
20	354	0.994	2
24	381	1.000	0



Strengths

- **Accurate with a tiny budget.** 92.4 % accuracy and 0.923 macro-F1 on unseen subjects with 8,862 parameters (44 % of the allowed budget), using raw signals only, with no feature engineering.
- **Coarse activity regime is rarely wrong.** Only 9 of 224 errors (4.0 %) cross between static and dynamic activities, so "is the person moving?" is much more reliable than the 6-way label (a property of this 6-class model; no dedicated binary detector was trained).
- **LAYING is near-perfect** (precision 0.994, recall 1.00): the gravity direction in `total_acc` is unambiguous.
- **Dynamic classes have high recall** (WALKING 0.95, UPSTAIRS 0.97, DOWNSTAIRS 0.91) and the model is cheap (0.42 MMAC/window).

Failure cases

1. **SITTING vs STANDING** is the dominant failure (73 + 62 = 135 of 224 errors). Both are near-static with similar waist-phone orientation; the only cue is a small tilt difference that varies between people. It is concentrated in a few subjects (see the per-subject table): subject 10 reaches only about 69 % accuracy (48 of the 135 sitting/standing errors are his or hers alone) and subjects 9 and 2 are also below 90 %, while subjects 12, 20 and 24 are at or near 99-100 %. The three worst subjects account for 173 of the 224 errors.
2. **Walking variants** (77 errors): DOWNSTAIRS is confused with UPSTAIRS (27) and WALKING with DOWNSTAIRS (26). The model sees gait rhythm well but stair direction less reliably; a 2.56 s window may contain too few distinctive impact events.
3. **Subject-dependence.** Per-subject accuracy ranges from about 0.69 to 1.00, so the aggregate number hides large between-person differences.

Limitations of the study

- **Validation-to-test gap.** Validation macro-F1 was 0.990 (4 subjects) versus 0.923 on test (9 subjects). Four validation subjects are a small and optimistic sample of between-subject variability; the per-subject spread above is consistent with this, but I did not run an experiment that isolates the cause. A leave-subjects-out cross-validation would give a more honest model-selection signal.
- **Selection on one split, 3 seeds.** The A-vs-C difference (0.0008) is inside seed noise; the ranking of A and C is not statistically established.
- **Model D was only evaluated on validation data**, never on the test set, so no test comparison between C and D exists.
- **Windowing and non-causality.** Pre-segmented overlapping windows are a dataset convenience, and A/B/C use symmetric padding, so they cannot be run sample-by-sample on a stream without redesign (D addresses this, at an accuracy cost).
- **No hardware evidence.** MACs, memory and receptive fields are derived from the architecture; nothing was quantised or measured on a device.

7. Additional validation-only analyses

These were run on the training subjects only (the test set was never touched) and help interpret the design choices.

7.1 Which sensor matters? (Model-A architecture, 3 seeds each)

```
ab = ext["sensor_ablation"]
t = pd.DataFrame({k: {"channels": len(v["channels"]), "params": v["params"], "mean val macro-F1":
round(v["mean_val_macro_f1"], 4),
                        "std": round(v["std_val_macro_f1"], 4)} for k, v in ab.items()}).T
display(t)
```

	channels	params	mean val macro-F1	std
acc_gyro_full	6.0	3766.0	0.9890	0.0006
acc_only	3.0	3334.0	0.9684	0.0039
gyro_only	3.0	3334.0	0.7338	0.0011
body_acc_gyro	6.0	3766.0	0.8485	0.0085

- **Accelerometer-only keeps most of the performance** (0.968), **gyroscope-only does not** (0.734): gravity direction, not rotation, carries the posture information.
- Replacing total_acc by gravity-free body_acc drops the score to 0.849, which supports the input choice in Section 1.

7.2 Per-subject variability on validation subjects

Even on the four validation subjects the score varies by subject (e.g. subject 25 is the hardest for C and D), which foreshadows the larger spread seen on the test subjects.

```
rows = []
for m in ["A", "C", "D", "Linear"]:
    d = ext["per_subject"][m]["per_subject"]
    rows.append({"Model": m, **{f"subj {s['subject_id']}": round(s["macro_f1"], 3) for s in d}})
display(pd.DataFrame(rows).set_index("Model"))
```

	subj 1	subj 3	subj 25	subj 27
Model				
A	0.999	0.976	0.982	0.999
C	1.000	0.989	0.972	1.000
D	0.996	0.991	0.930	0.998
Linear	0.548	0.657	0.548	0.679

7.3 Implications for low-power, always-on use (derived, not measured)

- Model C at the UCI HAR hop of 64 samples (1.28 s) needs about 0.32 MMAC/s and 34.6 KB of fp32 weights (about 8.7 KB at int8, before metadata).
- Model A needs 57 % fewer parameters and 34 % fewer MACs than C for a validation difference of 0.0008, so it is the natural candidate for a wake-up-style first stage; D is the natural starting point for a true streaming implementation because it is causal and has no pooling.
- None of this was quantised or run on hardware.

8. Conclusions and next steps

A compact depthwise-separable CNN (Model C, 8,862 parameters) classifies raw IMU windows from unseen subjects with **92.4 % accuracy and 0.923 macro-F1**. Its errors are structured: nearly all stay within the static or the dynamic group, and the dominant failure is SITTING versus STANDING, concentrated in a few subjects. The Pareto analysis shows that more capacity (B) did not help, that a causal design (D) costs some accuracy, and that A offers almost the same validation accuracy at much lower cost.

If continued: leave-subjects-out cross-validation for model selection; evaluating A and D on the test set under the same single-evaluation rule; subject-wise augmentation (orientation jitter) to attack the sitting/standing ambiguity; post-training int8 quantisation and a causal/streaming port of C or D validated against the PyTorch reference; measurements on target hardware.

Appendix: reproducibility

```
pip install -r requirements.txt
python scripts/download_data.py --out_dir data
python scripts/run_experiments.py --config configs/baseline.yaml      # Phase 1 (A/B/C x 3 seeds)
python scripts/train_final.py --config configs/baseline.yaml         # Phase 2 (17 epochs)
python scripts/evaluate_final.py --config configs/baseline.yaml       # single test evaluation (PyTorch)
python scripts/run_research_extension.py --config configs/baseline.yaml # Model D, Linear, ablation, Pareto (validation only)
```

`scripts/evaluate_final_numpy.py` is a torch-free equivalent of the evaluation script. It was used to produce the numbers above because PyTorch was not available where the report was assembled; it reproduces all 2,947 PyTorch predictions of a previous checkpoint exactly (`--verify_against`).